

Build night 1

Data Collection and Risk Hook

What a trading agent needs to see, and the code that can say no.

Travis Sim

NUS · Singapore

Thu 22 Oct 2026 · COM3/COM4

Where we are

Tonight's question: which data would hurt your agent most if it went stale?

DONE

Workshop 1

Tue 13 Oct

Picked a trading bot and built a first one

TONIGHT

Build night 1

Thu 22 Oct

Data collection and the risk hook

NEXT

Build night 2

Thu 29 Oct

Data bars and change alerts

Tonight

18:00 to 20:00, mostly hands on.

18:00 — 10 min
Progress circle
last result and tonight's question

18:10 — 10 min
Mini-talk
four kinds of data, the risk hook

18:20 — 20 min
Demo tour
you predict, then we run it

18:40 — 20 min
Guided mission
run it, change one thing, chart it

19:00 — 50 min
Free build
your own project

19:50 — 10 min
Show and tell
push, one line on next step

A smart agent with bad data

The agent is the same in both cards. Only the input changed.

FRESH INPUT

Price read 1 second ago

106.60

USD per SOL

Agent: "Buy a small size."

A sensible answer.

ONE BAD INPUT

Price read 4 minutes ago

106.60

USD per SOL

Agent: "Buy a small size."

Same confident answer, now wrong.

Illustrative

Four kinds of data

Every agent decision reads some mix of these.

1 Venue price
`jupiterQuote`

DATA FAMILY 1 OF 4

Venue price

What you can actually trade at, right now.

ON SOLANA

A Jupiter quote: sell 1 SOL, get N USDC, routed across the pools.

HOW IT GOES BAD

Moves every second. One venue's price is not the market.

Four kinds of data

Every agent decision reads some mix of these.

1 Venue price
`jupiterQuote`

2 Oracle price
`pythOnChain`

DATA FAMILY 2 OF 4

Oracle price

A reference price that lives on chain.

ON SOLANA

A Pyth price account: price, confidence band, publish time.

HOW IT GOES BAD

Can be old, or wide. Check its age and confidence before you trust it.

Four kinds of data

Every agent decision reads some mix of these.

1 Venue price
`jupiterQuote`

2 Oracle price
`pythOnChain`

3 Ordering
`slot numbers`

DATA FAMILY 3 OF 4

Ordering

What happened in which slot.

ON SOLANA

A slot is the chain's clock, about 0.4 seconds. Compare postedSlot with currentSlot.

HOW IT GOES BAD

Two prices from different slots are not from the same moment.

Four kinds of data

Every agent decision reads some mix of these.

1. Venue price — `jupiterQuote`
2. Oracle price — `pythOnChain`
3. Ordering — `slot numbers`
4. Inventory — `balances`

DATA FAMILY 4 OF 4

Inventory

What you hold right now.

ON SOLANA

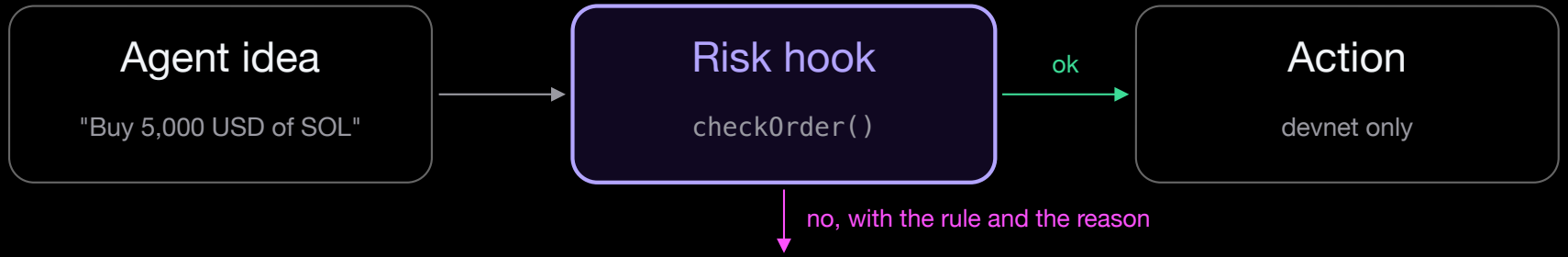
Token balances. SOL is counted in lamports: 1 SOL is 1,000,000,000 lamports.

HOW IT GOES BAD

Wrong unit or an old balance, and your size is wildly off.

The risk hook

Plain code between the idea and the action. It can say no, and say why.



THE FOUR RULES IN THE SAMPLE CODE

1,000 USD	50 bps	60 seconds	sizes make sense
max notional	max oracle gap	max oracle age	unit sanity

Real numbers

SOL/USDC from three places, same moment.

JUPITER QUOTE

108.62

USD per SOL

sell 1 SOL, get USDC

PYTH ORACLE

108.61

USD per SOL

+/- 2.7 bps confidence, 0 slots old

ORCA SOL/USDC POOL

108.59

USD per SOL

pool price from its state

+1.8 bps Jupiter vs Pyth

-1.7 bps Orca vs Pyth

+3.5 bps Jupiter vs Orca

Reads use public mainnet data; every write is devnet only; no real funds, no seed phrases.

Source: recorded snapshot (code/recorded/snapshot.json), captured 2026-10-21 17:53:59Z. Jupiter quote API, Pyth account and Orca pool read over public mainnet RPC. A bps is one hundredth of a percent.

A collector is short

One function, one data family.

src/collectors/jupiter.ts

```
/** USDC has 6 decimals, so outAmount / 1e6 is the USD received for 1 SOL. */
export function priceFromQuote(quote: { outAmount: string }): number {
  return Number(quote.outAmount) / 1e6;
}
export async function jupiterQuote(): Promise<VenueQuote> {
  const quote = (await getJson(JUPITER_URL)) as { outAmount?: string };
  if (!quote.outAmount) throw new Error("Jupiter quote had no outAmount");
  return {
    id: "jupiter",
    // ...label, ts, source
    price: priceFromQuote({ outAmount: quote.outAmount }),
  };
}
```

1 **No key, no account**

Public quote API for 1 SOL to USDC.

2 **Watch the unit**

USDC has 6 decimals: divide by 1e6.

3 **Fail loudly**

No quote means an error, never a made-up price.

Source: demo-repo/code/src/collectors/jupiter.ts (excerpt, fields elided)

Demo: the price gap

PREDICT FIRST

Do the three prices agree? By how many bps do they differ?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- Jupiter, Pyth and Orca, one line each
- the gap in bps between each pair

Fallback: Recorded data is the default. No network needed.



Try it: nus-solana-demos.vercel.app/0002/02-price-gap.html

Live mode: switch it on to watch real Solana data.

Recorded data. Runs in your browser.

Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data)

Demo: the risk hook

PREDICT FIRST

Which orders pass, and which rule stops each one that does not?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- accepted: buy 250 USD, within all limits
- rejected: buy 5,000 USD, max-notional
- each rejection names its rule and says why

Fallback: Same recorded data. The decisions are the same every run.



Try it: nus-solana-demos.vercel.app/0002/03-risk-hook.html

Live mode: switch it on to watch real Solana data.

Recorded data. Runs in your browser.

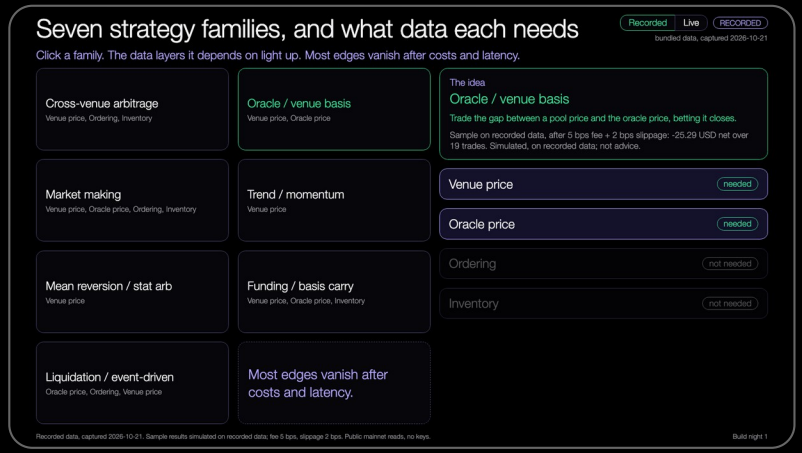
Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data)

The strategy map

Seven families. Each one needs certain data.

V venue O oracle S slot (ordering) I inventory

	V	O	S	I
Cross-venue arbitrage	●	○	●	●
Oracle vs venue basis	●	●	○	○
Market making	●	●	●	●
Trend / momentum	●	○	○	○
Mean reversion / stat arb	●	○	○	○
Funding / basis carry	●	●	○	●
Liquidation / event-driven	●	●	●	○



Pick a family: its data lights up.

Most edges vanish after costs and latency.

`Try it: nus-solana-demos.vercel.app/0002/04-strategy-map.html`

Live mode: switch it on to watch real Solana data.

Source: data each family needs is our teaching map, not a recommendation. Simulated, on recorded data.

Fees and slippage

A gross edge is not what you keep.

Gross edge	what the chart shows
minus fees	5 bps on each leg
minus slippage	2 bps on each leg
Net result	what you keep

RECORDED RUN, ONE STRATEGY

Pool vs oracle basis	gross +1.31 USD net -25.29 USD



Drag the sliders: gross bars shrink into net.

Simulated, on recorded data; not advice.

`Try it: nus-solana-demos.vercel.app/0002/05-fees-slippage.html`

Live mode: switch it on to watch real Solana data.

Source: Learning Lab demo and code/out/strategies.json, recorded data. Simulated, on recorded data; not advice.

Try the demos yourself

OPEN ON YOUR LAPTOP

`nus-solana-demos.vercel.app/0002`

Open it on your laptop and move the sliders.

Five demos: data layers, price gap, risk hook, strategy map, fees and slippage. Recorded data; nothing to install.



Scan to open the demos

Your mission

18:40 to 19:00. Twenty minutes.

1

Run

Run the sample code on recorded data

2

Predict

Guess what the risk hook will do before you look

3

Change

Change one limit, or add fees and slippage to a strategy

4

Chart

Chart the result, before and after

Finish line: one chart with a before and an after.

Build menu

19:00 to 19:50. Pick one, or bring your own project.

BEGINNER	Compare two prices and flag the gap
BEGINNER	Make the project run fully offline
STANDARD	Test one strategy family with fees and slippage
STANDARD	Write a new risk rule
ADVANCED	Add a data source

Three takeaways

- [ ] One bad input beats a smart agent.
- [ ] Four kinds of data: venue price, oracle price, ordering, inventory.
- [ ] The risk hook is plain code that can say no, and say why.

Branch out

After tonight, in your own time.

BEGINNER

Change a limit and watch which orders flip from rejected to accepted.

STANDARD

Run a second strategy from the map with honest costs and compare the charts.

ADVANCED

Add a new pool or venue collector, or a staleness check that fires on old oracle data.

Share back

Before you leave, and by next Thursday.

TONIGHT

1

Push your branch to your own repo.

THU 29 OCT

2

Bring your chart.

ANY TIME

3

Send your repo link to the leads.

Get the code

REPO

`github.com/travisim/solana-agentic-trading-nus`

`folder: 0002-data-collection-risk-hook/demo`

```
1  $ npm install
```

```
2  $ npm run demo -- --recorded
```

Run both from the demo/code folder. Node 24. No keys, no AI credits. Recorded data is bundled.



Scan for tonight's demos and code

Let's connect



LinkedIn

linkedin.com/in/travisim



Telegram

t.me/travisim


SOLANA